

FRAUDULENT CLAIM DETECTION

Ganesh Babu Kamma
Harshit Kumar Vaish

CONTENTS

- Problem Statement
- Data Dictionary
- Data Preparation
- Univariate Analysis
- Correlation Analysis
- Class Imbalance
- Bivariate Analysis
- Feature Engineering
- Logistic Regression
- Random Forest
- Prediction and Model Evaluation
- Summary/Conclusion



PROBLEM STATEMENT

Global Insure, a leading insurance company, processes thousands of claims annually. However, a significant percentage of these claims turn out to be fraudulent, resulting in considerable financial losses. The company's current process for identifying fraudulent claims involves manual inspections, which is time-consuming and inefficient. Fraudulent claims are often detected too late in the process, after the company has already paid out significant amounts. Global Insure wants to improve its fraud detection process using data-driven insights to classify claims as fraudulent or legitimate early in the approval process. This would minimise financial losses and optimise the overall claims handling process.



BUSINESS OBJECTIVE

Global Insure wants to build a model to classify insurance claims as either fraudulent or legitimate based on historical claim details and customer profiles. By using features like claim amounts, customer profiles and claim types, the company aims to predict which claims are likely to be fraudulent before they are approved.

DATA DICTIONARY

Column Name	Description
months_as_customer	Represents the duration in months that a customer has been associated with the insurance company.
age	Represents the age of the insured person.
policy_number	Represents a unique identifier for each insurance policy.
policy_bind_date	Represents the date when the insurance policy was initiated.
policy_state	Represents the state where the insurance policy is applicable.
policy_csl	Represents the combined single limit for the insurance policy.
policy_deductable	Represents the amount that the insured person needs to pay before the insurance coverage kicks in.
policy_annual_premium	Represents the yearly cost of the insurance policy.
umbrella_limit	Represents an additional layer of liability coverage provided beyond the limits of the primary insurance policy.
insured_zip	Represents the zip code of the insured person.
insured_sex	Represents the gender of the insured person.
insured_education_level	Represents the highest educational qualification of the insured person.
insured_occupation	Represents the profession or job of the insured person.
insured_hobbies	Represents the hobbies or leisure activities of the insured person.
insured_relationship	Represents the relationship of the insured person to the policyholder.
capital-gains	Represents the profit earned from the sale of assets such as stocks, bonds, or real estate.
capital-loss	Represents the loss incurred from the sale of assets such as stocks, bonds, or real estate.
incident_date	Represents the date when the incident or accident occurred.
incident_type	Represents the category or type of incident that led to the claim.
collision_type	Represents the type of collision that occurred in an accident.
incident_severity	Represents the extent of damage or injury caused by the incident.
authorities_contacted	Represents the authorities or agencies that were contacted after the incident.
incident_state	Represents the state where the incident occurred.
incident_city	Represents the city where the incident occurred.
incident_location	Represents the specific location or address where the incident occurred.
incident_hour_of_the_day	Represents the hour of the day when the incident occurred.
number_of_vehicles_involved	Represents the total number of vehicles involved in the incident.
property_damage	Represents whether there was any damage to property in the incident.
bodily_injuries	Represents the number of bodily injuries resulting from the incident.
witnesses	Represents the number of witnesses present at the scene of the incident.
police_report_available	Represents whether a police report is available for the incident.
total_claim_amount	Represents the total amount claimed by the insured person for the incident.
injury_claim	Represents the amount claimed for injuries sustained in the incident.
property_claim	Represents the amount claimed for property damage in the incident.
vehicle_claim	Represents the amount claimed for vehicle damage in the incident.
auto_make	Represents the manufacturer of the insured vehicle.
auto_model	Represents the specific model of the insured vehicle.
auto_year	Represents the year of manufacture of the insured vehicle.
fraud_reported	Represents whether the claim was reported as fraudulent or not.
_c39	Represents an unknown or unspecified variable.

DATA PREPARATION

- Initial Data Overview
- Dataset loaded with 1000 claims and 40 features
- Examined features, Data (Numbers, date, claim amount), and basic statistics to understand the claims data.
- Identified key columns like insured hobbies , incident state , auto model.
- Loading the necessary libraries for analysis

DATA PREPARATION

- Fixing Data Type

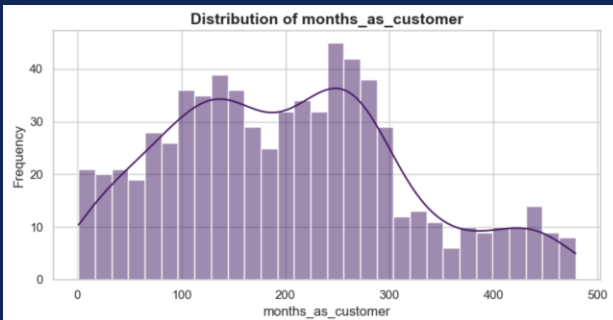
- Identified the column with different data type, as Date columns were marked as Object type.
- Converted 'policy_bind_date' and 'incident_date' to date time format using `pd.to_datetime` function.
- Category type columns were stored as Object type so they were converted to category type to improve computation speed.

UNIVARIATE ANALYSIS

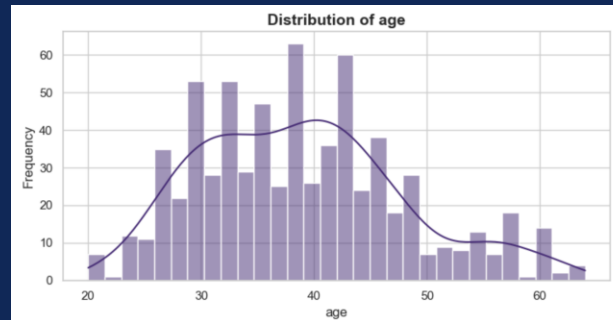
- Numerical Column Visualisation
 - Identified the Numerical Columns from the cleaned dataframe.
 - Represented them into histogram chart.
 - Used matplotlib and seaborn for plotting these graphs.
 - Performed correlation analysis and class balance.
 - Separated numerical and categorical columns.

UNIVARIATE ANALYSIS

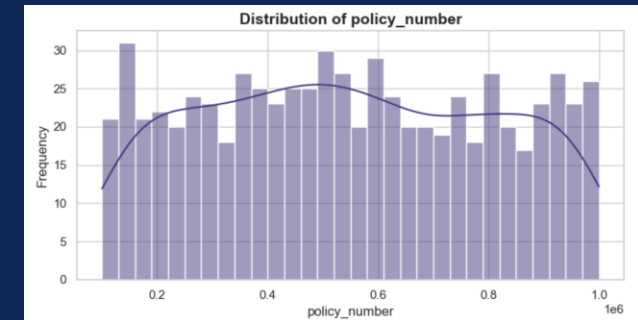
**Distribution of
month_as_customer**



Distribution of age

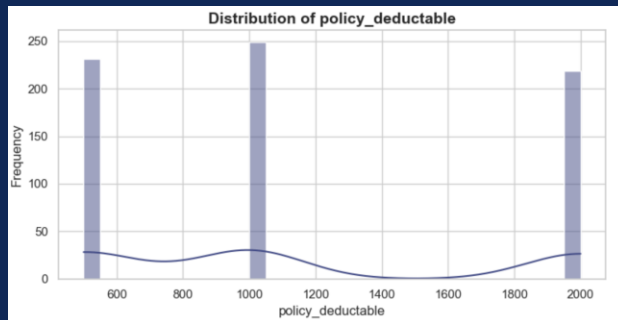


**Distribution of Policy
Number**

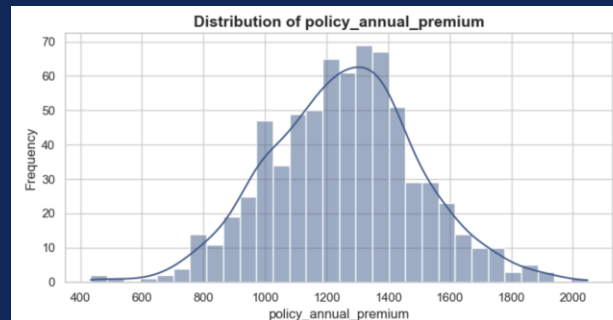


UNIVARIATE ANALYSIS

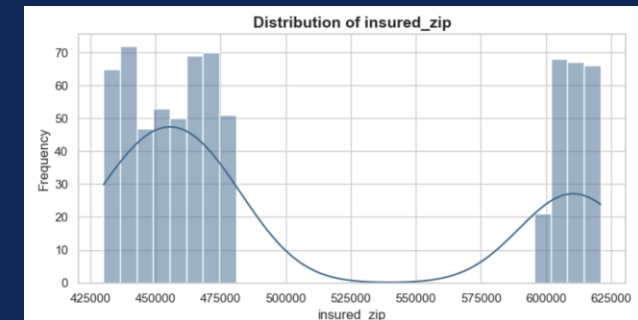
Distribution of policy_deductable



Distribution of policy_annual_premium

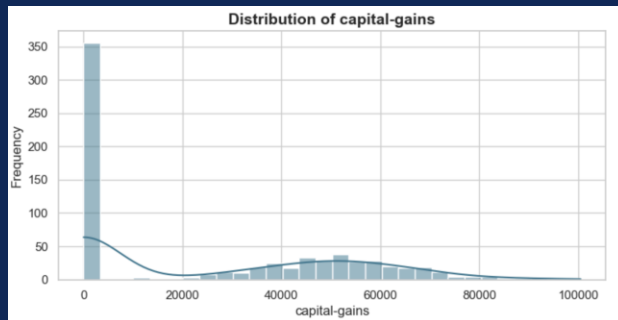


Distribution of Insured_zip

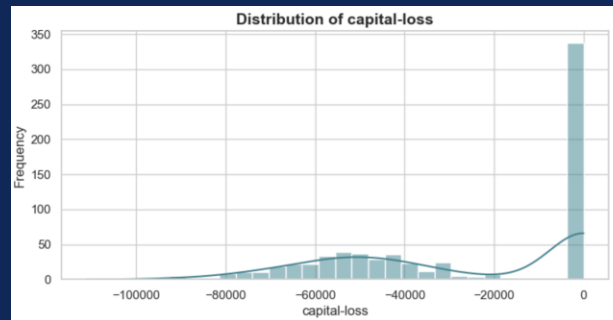


UNIVARIATE ANALYSIS

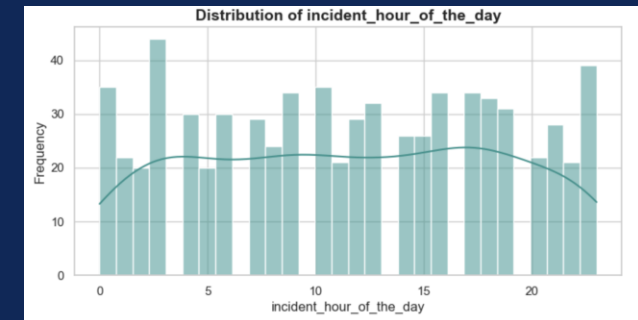
Distribution of capital_gains



Distribution of capital_loss

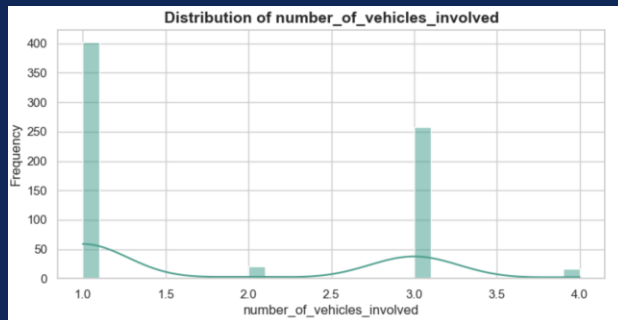


Distribution of Incident_hour_of_the_day

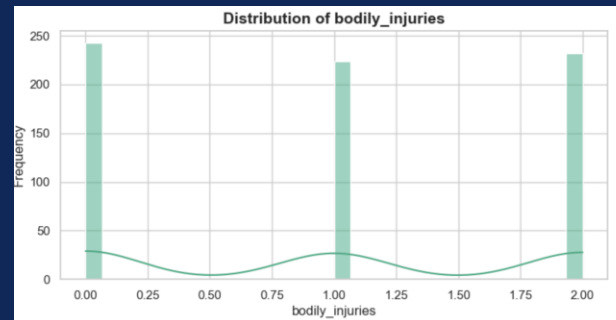


UNIVARIATE ANALYSIS

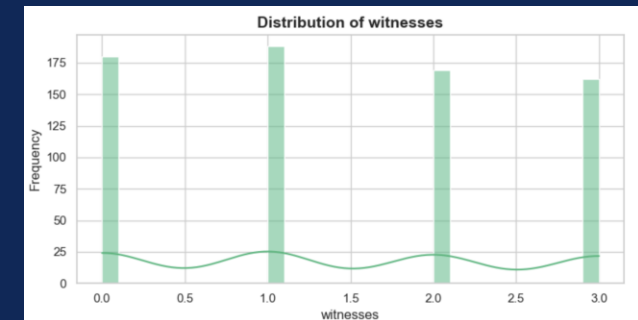
Distribution of number_of_vehicles_involved



Distribution of bodily_injuries

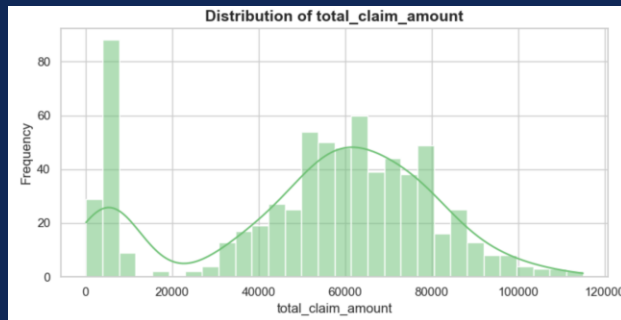


Distribution of witnesses

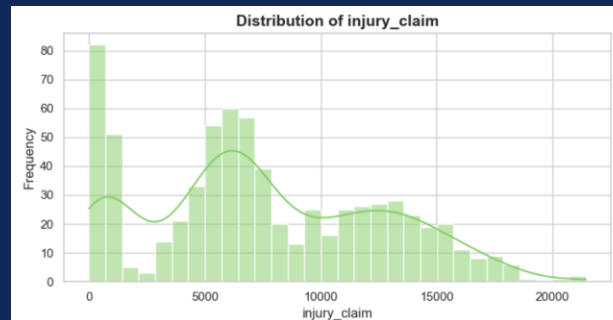


UNIVARIATE ANALYSIS

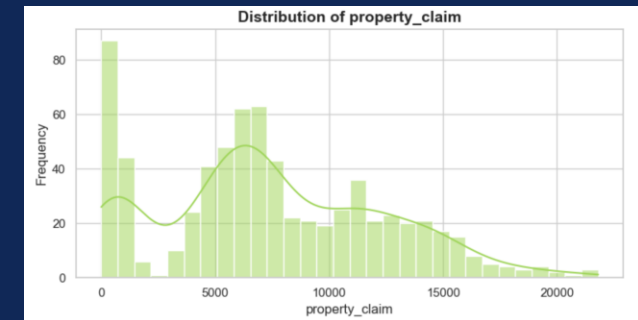
Distribution of total_claim_amount



Distribution of injury_claim

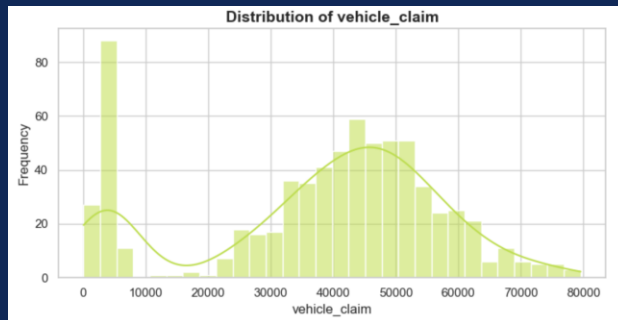


Distribution of property_claim

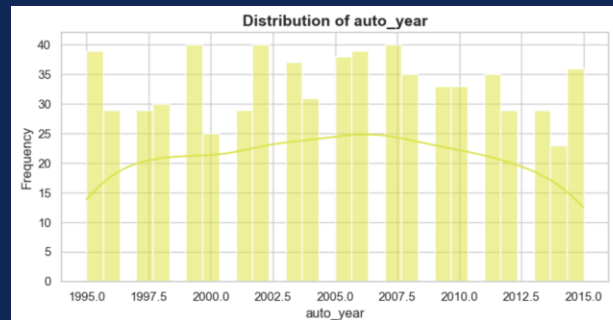


UNIVARIATE ANALYSIS

Distribution of vehicle_claim

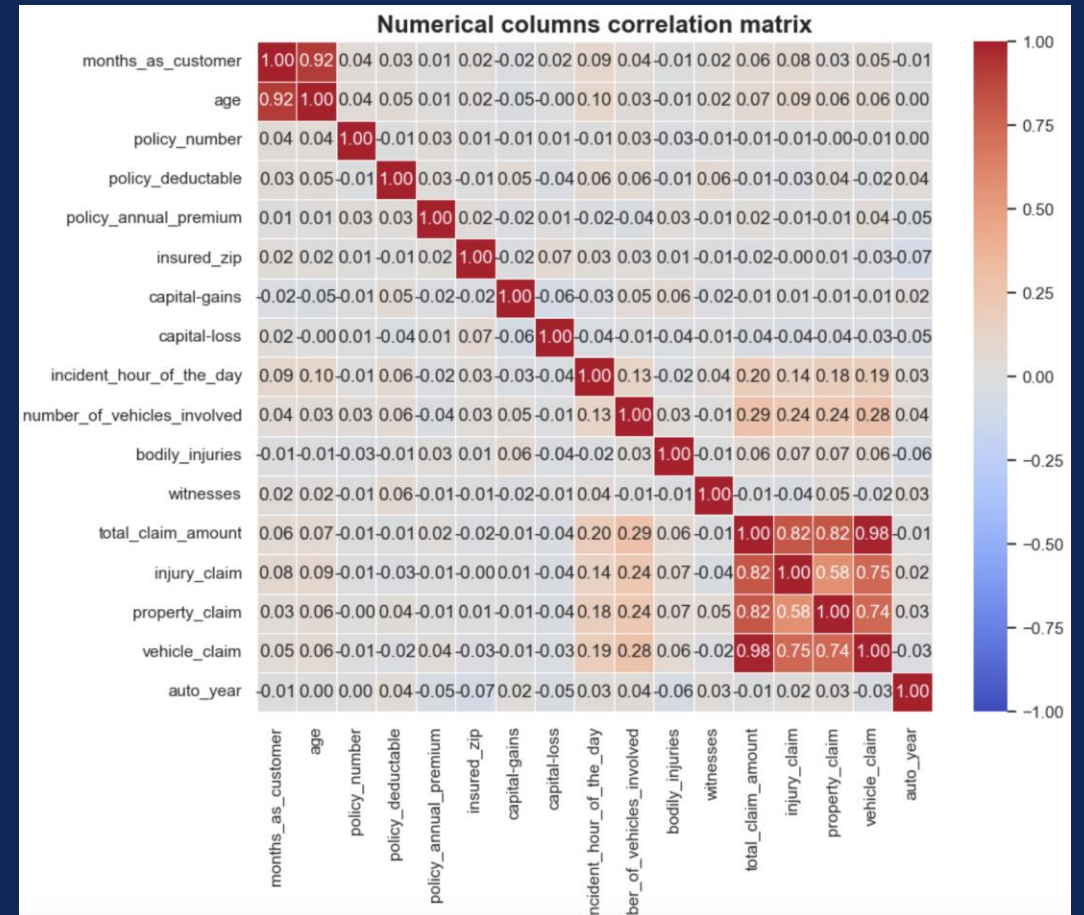


Distribution of auto_year



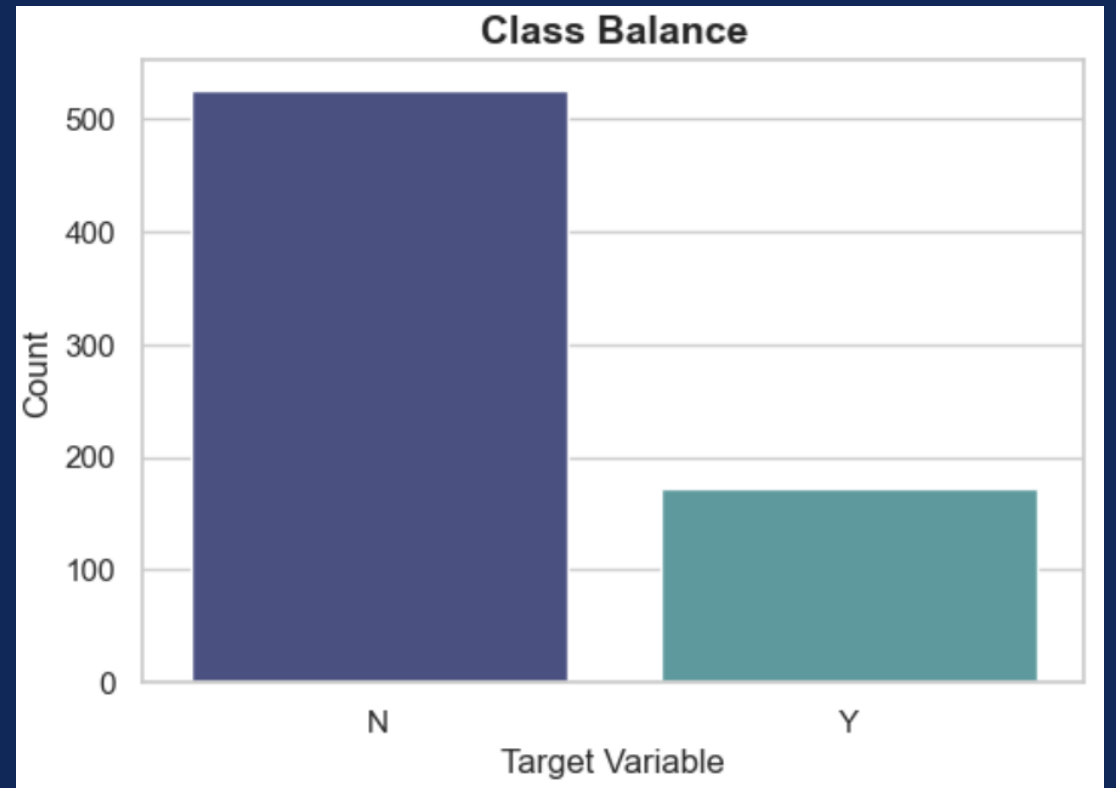
UNIVARIATE ANALYSIS

○ Correlation analysis



UNIVARIATE ANALYSIS

- Class balance

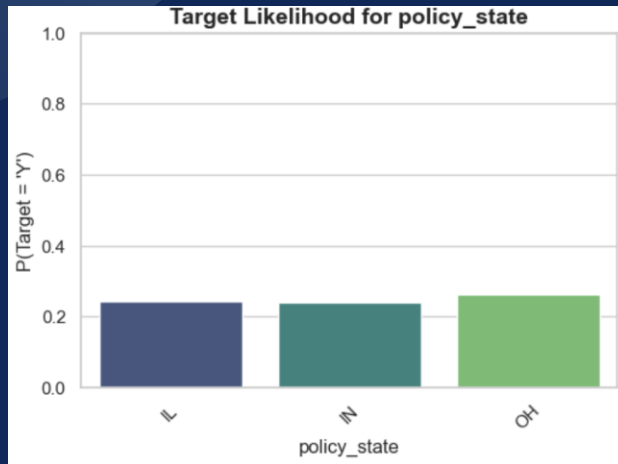


BIVARIATE ANALYSIS

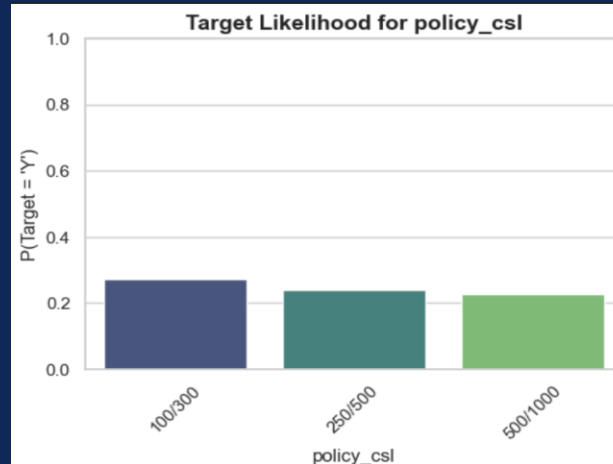
- Categorical Column Visualisation
 - Identified the Categorical Columns from the dataframe.
 - Represented them into Bar chart.
 - Used matplotlib and seaborn for plotting these graphs.
 - Performed likelihood analysis of the target variable based on the categorical features.

BIVARIATE ANALYSIS

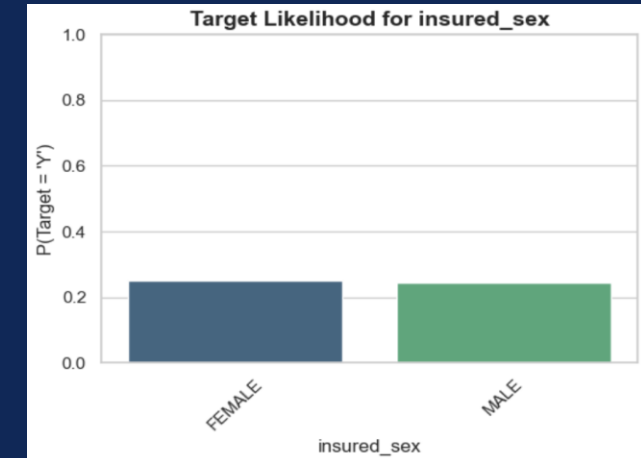
Target Likelihood for policy_state



Target Likelihood for policy_csl

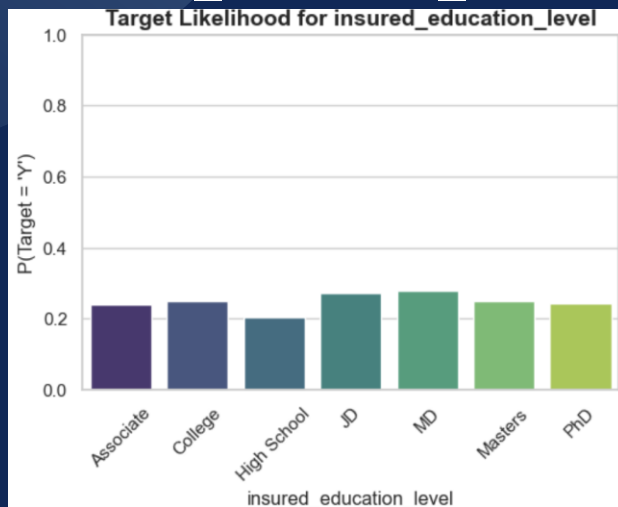


Target Likelihood for insured_sex

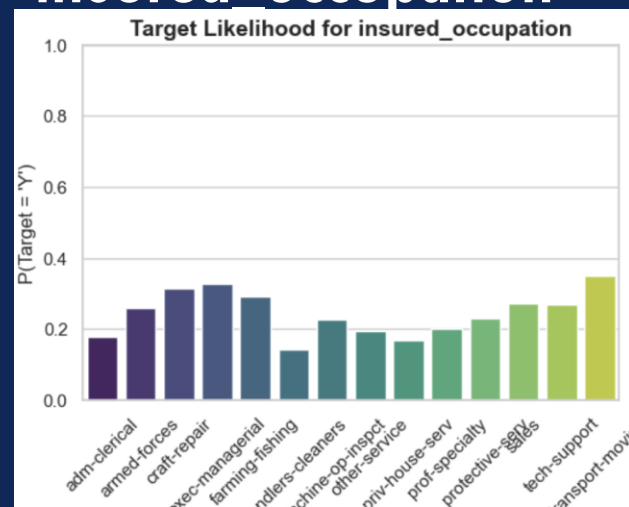


BIVARIATE ANALYSIS

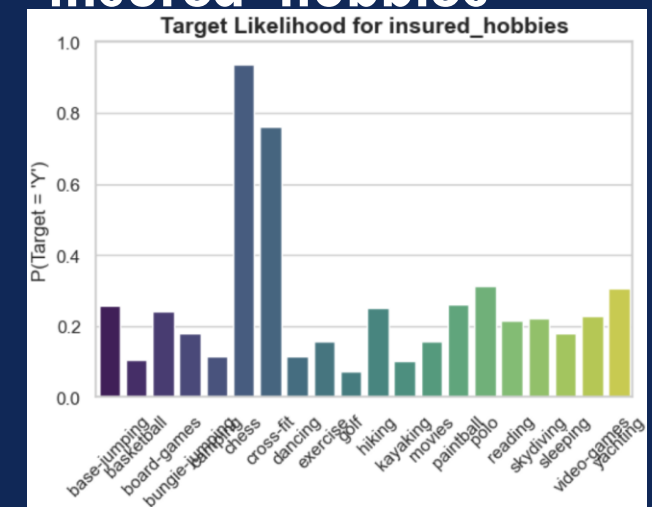
Target Likelihood for
insured_education_level



Target Likelihood for
insured_occupation

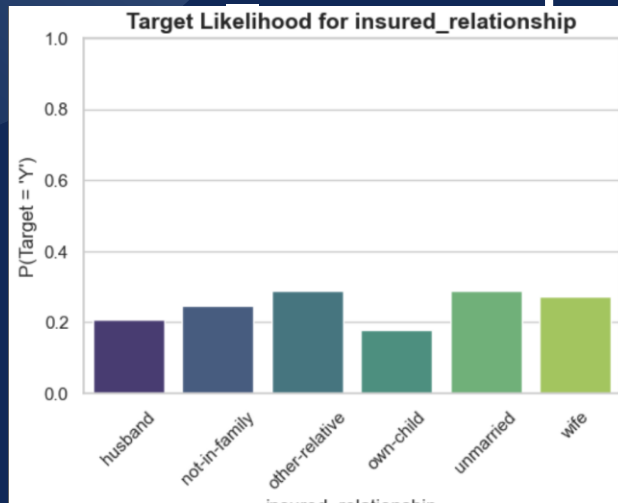


Target Likelihood for
insured_hobbies

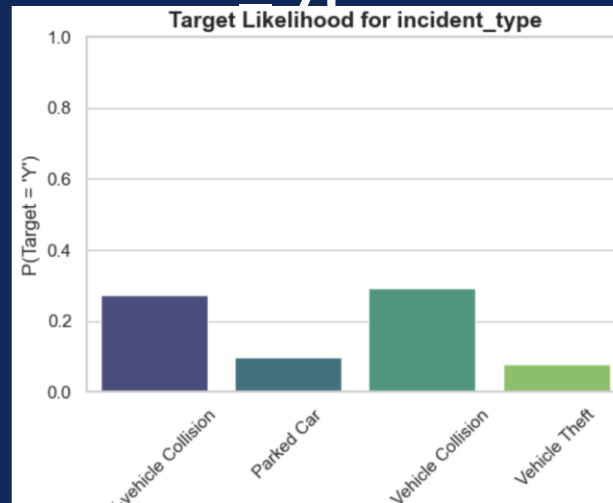


BIVARIATE ANALYSIS

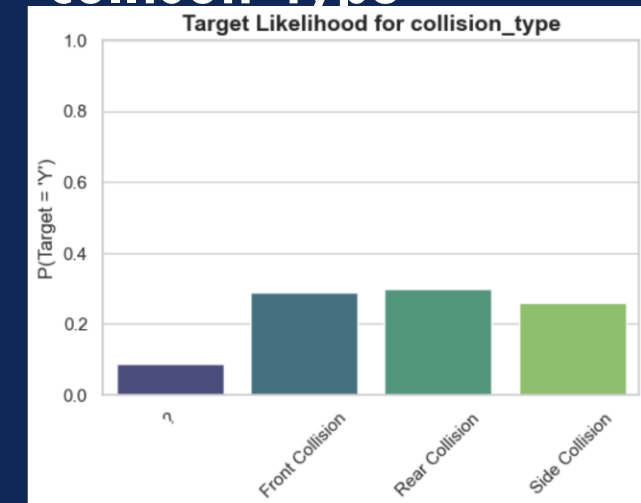
Target Likelihood for insured_relationship



Target Likelihood for incident_type

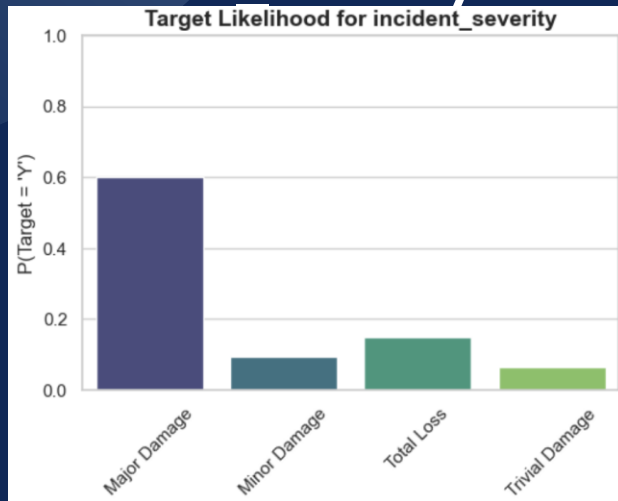


Target Likelihood for collision_type

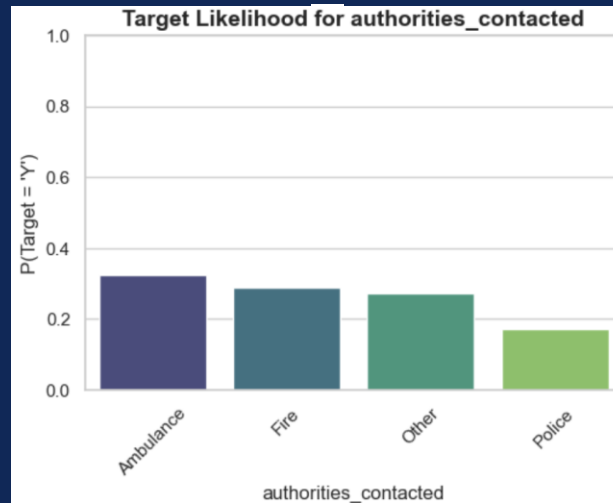


BIVARIATE ANALYSIS

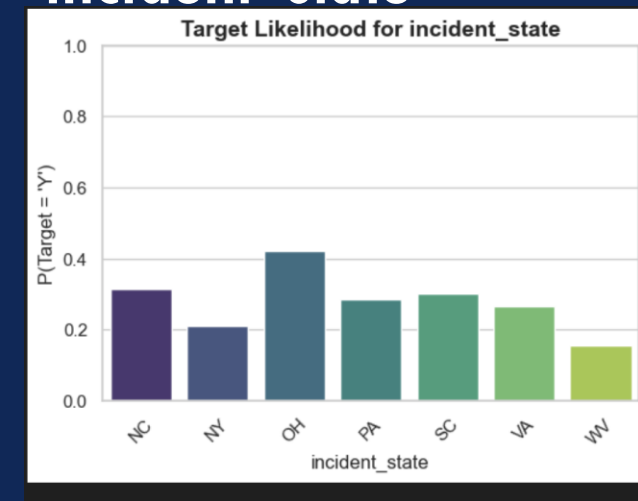
Target Likelihood for incident severity



Target Likelihood for authorities contacted

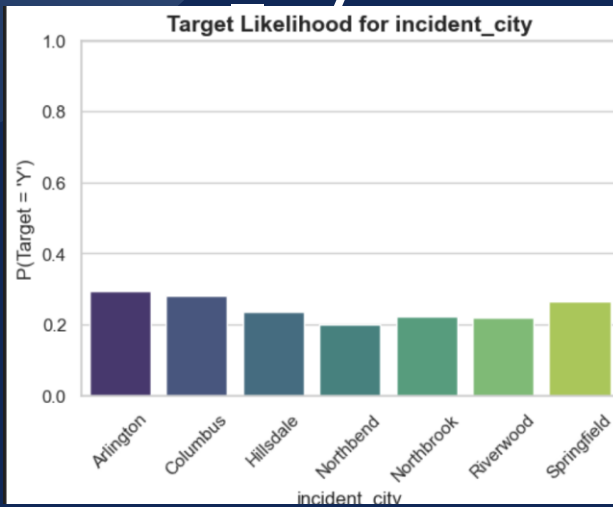


Target Likelihood for incident state

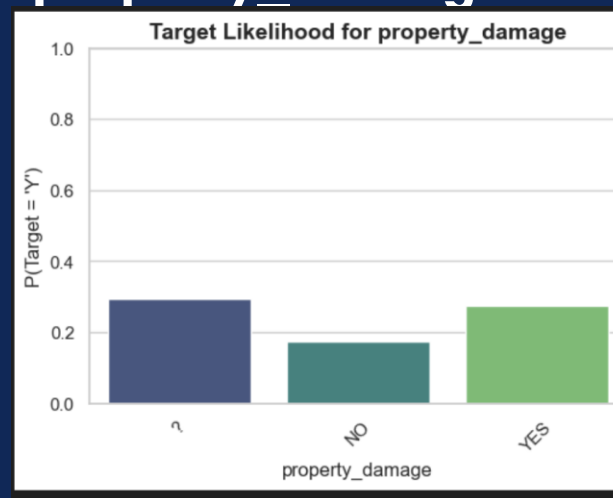


BIVARIATE ANALYSIS

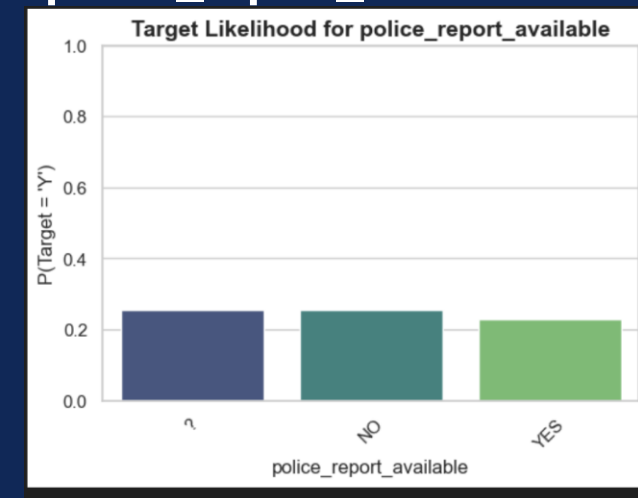
Target Likelihood for incident_city



Target Likelihood for property_damage

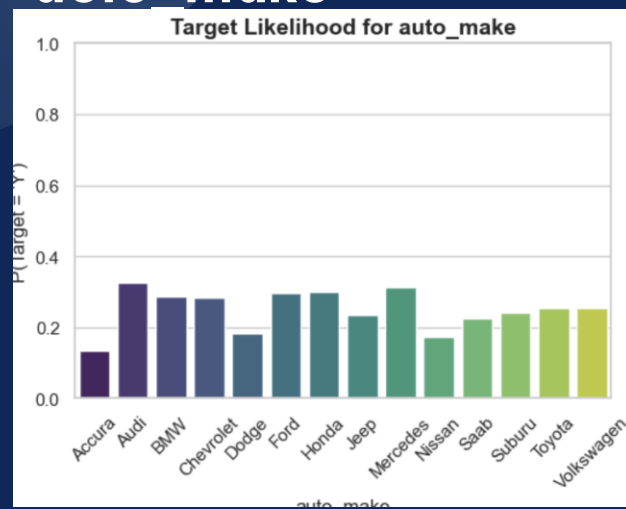


Target Likelihood for police_report_available

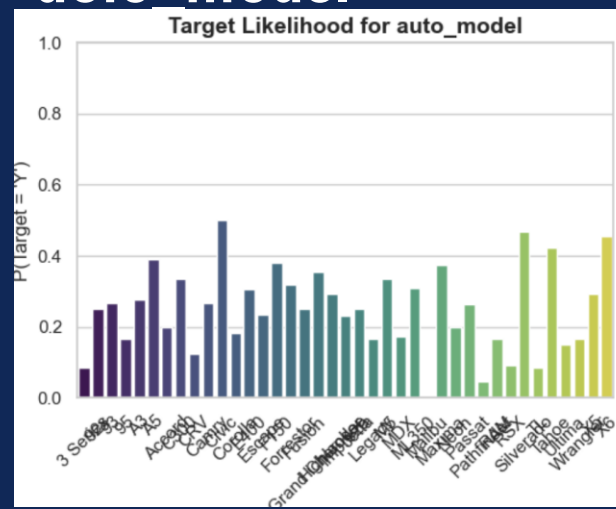


BIVARIATE ANALYSIS

Target Likelihood for
auto_make



Target Likelihood for
auto_model



BIVARIATE ANALYSIS

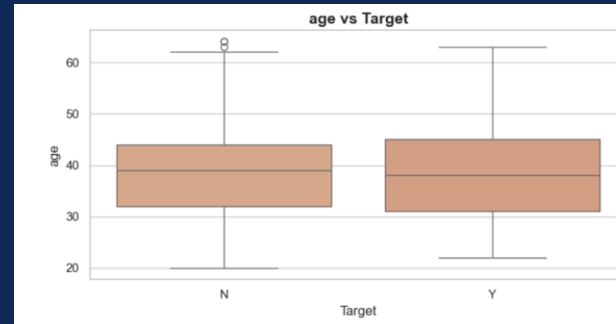
- Numerical Column Visualisation
 - Identified the Categorical Columns from the dataframe.
 - Represented them into Box plot chart.
 - Used matplotlib and seaborn for plotting these graphs.
 - Performed impact analysis of the target variable based on the Numerical features.

BIVARIATE ANALYSIS

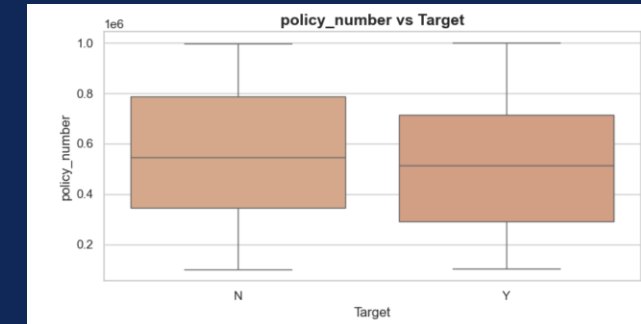
Month_as_customer vs Target



Age vs Target

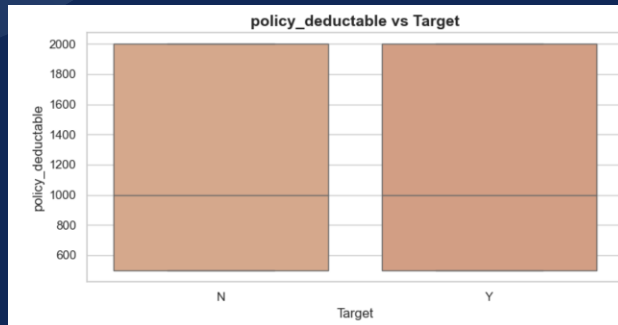


Policy vs Target



BIVARIATE ANALYSIS

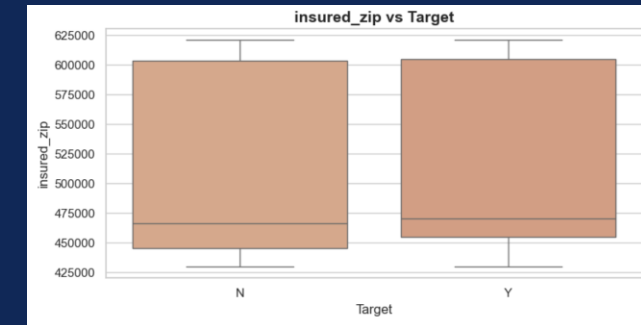
Policy_deductable vs Target



Policy_annual_premium vs Target

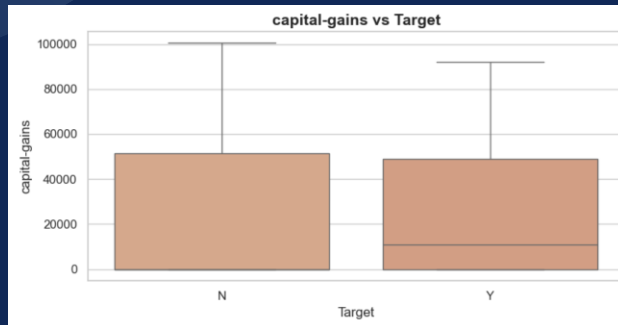


Insured_zip vs Target

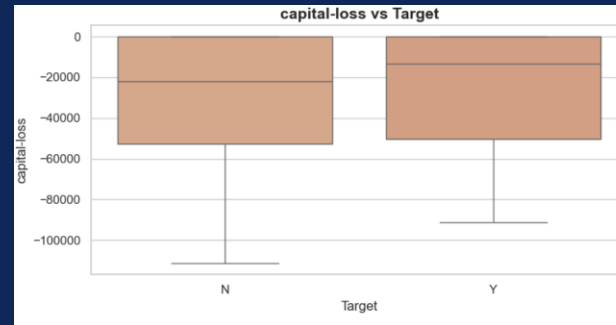


BIVARIATE ANALYSIS

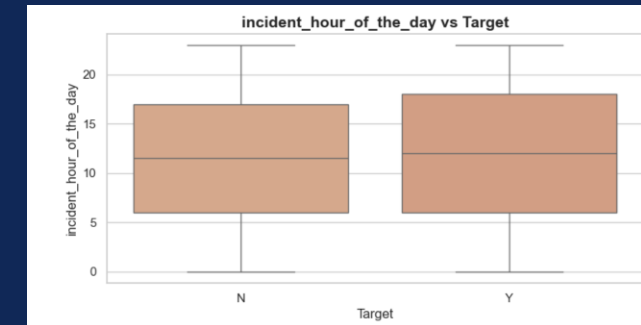
Capital_gains vs Target



Capital_loss vs Target

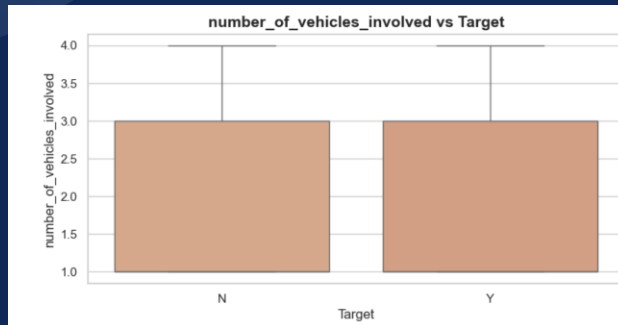


Incident_hour_of_the_day vs Target

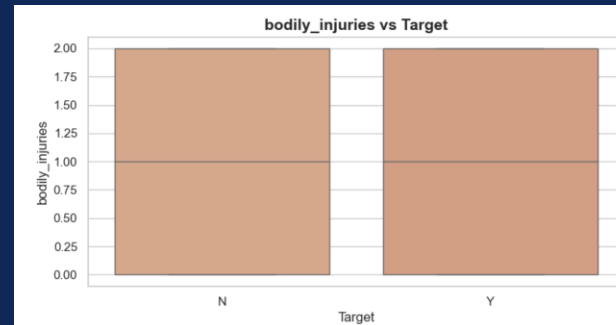


BIVARIATE ANALYSIS

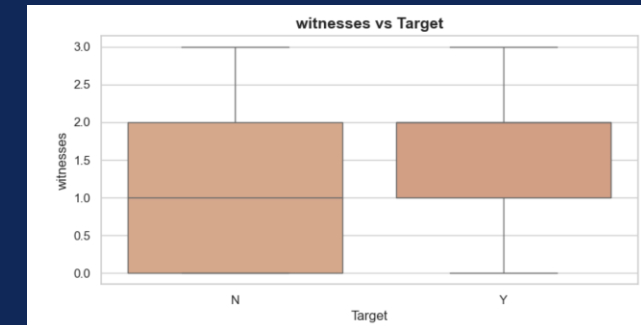
**Number_of_vehicles_
involved vs Target**



**Bodily_injuries vs
Target**



Witnesses vs Target



BIVARIATE ANALYSIS

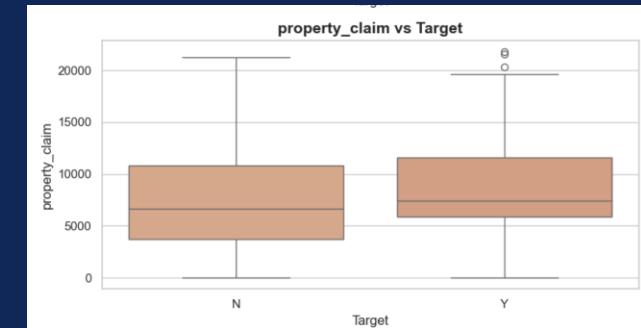
Total_claim_amount vs Target



Injury_claim vs Target

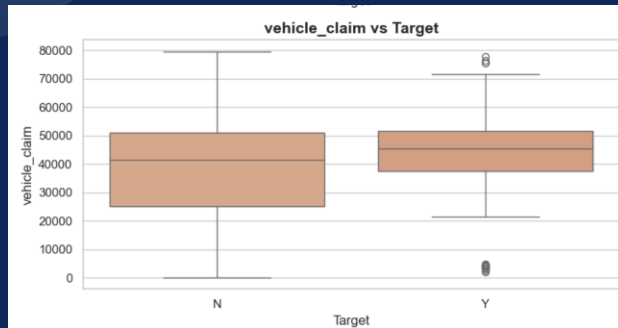


Property_claim vs Target

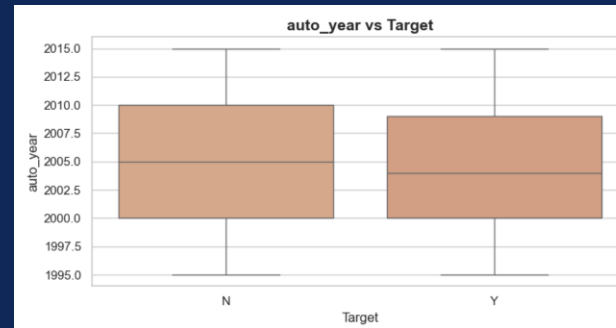


BIVARIATE ANALYSIS

Vehicle_claim vs Target



Auto vs Target



FEATURE ENGINEERING

- Resampling
 - Used `RandomOverSampler` to balance the data and handle class imbalance.
 - This helps prevent the model being biased toward the majority class.
 - Used the below code for resampling

```
# Import RandomOverSampler from imblearn library
from imblearn.over_sampling import RandomOverSampler

# Perform resampling on training data
ros=RandomOverSampler(sampling_strategy='auto',random_state=42)
X_train_resampled,y_train_resampled=ros.fit_resample(X_train,y_train)
```

FEATURE ENGINEERING

- Feature Creation
 - Created net_capital from capital_gains and capital_loss
 - Extracted month and day of week from incident_date.
 - Created high_gain_flag and high_loss_flag based on capital_gains and capital_loss
 - Used the below code for resampling

```
# Create new features based on your understanding for both training and validation data
for n_df in [X_train_resampled, X_test]:
    n_df['policy_age_days'] = (pd.to_datetime(n_df['incident_date']) - pd.to_datetime(n_df['policy_bind_date'])).dt.days
    n_df['incident_month'] = pd.to_datetime(n_df['incident_date']).dt.month
    n_df['incident_dow'] = pd.to_datetime(n_df['incident_date']).dt.dayofweek
    df['net_capital'] = df['capital-gains'] + df['capital-loss']
    df['high_gain_flag'] = (df['capital-gains'] > 0).astype(int)
    df['high_loss_flag'] = (df['capital-loss'] < 0).astype(int)
```


FEATURE ENGINEERING

- Redundant Columns
 - Removed the column which have high correlation.
 - Eliminated the column from which the data is extracted in previous step like policy_bind_date and incident_date.
 - Removed the column which do not influence the prediction.
- Combine values in categorical columns
 - Grouped together infrequent categories within the columns to reduce sparsity.
 - Handled missing values by replacing "?" entries in the columns.

FEATURE ENGINEERING

- Dummy Variable Creation
 - Created dummy variables for the train and test data.
 - Converted the categorical column into numerical using dummy variable creation.
 - Dropped the original columns after dummy variables are created.
- Feature Scaling
 - Used **StandardScaler** for standardizing the numerical columns.
 - Scaling is important as it brings all the Numerical column to the same scale.
 - Performed **fit_transform** on the train data whereas **transform** on the test data.

LOGISTIC REGRESSION MODEL

- Used RFECV for finding the optimal number of column required for building the logistic regression model.
- Used statsmodel for building the model got from the columns got from above step.
- Used the below code.

```
# Fit a logistic Regression model on X_train after adding a constant and output the summary
logm1 = sm.GLM(y_train_resampled,selected_df_constant, family = sm.families.Binomial())
logm1.fit().summary()
```

LOGISTIC REGRESSION MODEL

- Used VIF technique to identify the multicollinearity among the columns.
- Removed the columns whose p-value is above 0.05

Generalized Linear Model Regression Results							
Dep. Variable:	fraud_reported	No. Observations:	1052				
Model:	GLM	Df Residuals:	1029				
Model Family:	Binomial	Df Model:	22				
Link Function:	Logit	Scale:	1.0000				
Method:	IRLS	Log-Likelihood:	-370.68				
Date:	Wed, 13 Aug 2025	Deviance:	741.36				
Time:	01:10:57	Pearson chi2:	3.34e+03				
No. Iterations:	7	Pseudo R-squ. (CS):	0.4942				
Covariance Type:	nonrobust						
	coef	std err	z	P> z	[0.025	0.975]	
const	1.2814	0.188	6.834	0.000	0.914	1.649	
insured_education_level_MD	0.9646	0.277	3.487	0.000	0.422	1.507	
insured_occupation_transport-moving	0.8183	0.311	2.633	0.008	0.209	1.427	
insured_hobbies_chess	6.0883	0.745	8.175	0.000	4.629	7.548	
insured_hobbies_video-games	1.1930	0.399	2.993	0.003	0.412	1.974	
insured_relationship_not-in-family	1.0991	0.268	4.095	0.000	0.573	1.625	
insured_relationship_unmarried	0.8498	0.267	3.177	0.001	0.326	1.374	
collision_type_Side Collision	-0.5937	0.227	-2.616	0.009	-1.038	-0.149	
incident_severity_Minor Damage	-4.0080	0.277	-14.473	0.000	-4.551	-3.465	
incident_severity_Total Loss	-3.2092	0.260	-12.367	0.000	-3.718	-2.701	
incident_severity_Trivial Damage	-4.4687	0.550	-8.118	0.000	-5.548	-3.390	
incident_state_WV	-1.0617	0.269	-3.941	0.000	-1.590	-0.534	
incident_city_Northbrook	-0.9465	0.349	-2.713	0.007	-1.630	-0.263	
property_damage_NO	-0.5433	0.213	-2.553	0.011	-0.960	-0.126	
auto_make_Audi	1.5296	0.344	4.445	0.000	0.855	2.204	
auto_model_Civic	2.9836	0.628	4.749	0.000	1.752	4.215	
auto_model_F150	1.2443	0.545	2.284	0.022	0.177	2.312	
auto_model_Grand Cherokee	1.6171	0.563	2.872	0.004	0.514	2.721	
auto_model_M5	2.0633	0.857	2.407	0.016	0.383	3.744	
auto_model_Neon	0.9538	0.465	2.050	0.040	0.042	1.866	
auto_model_Pathfinder	-2.3733	0.720	-3.298	0.001	-3.783	-0.963	
auto_model_Silverado	1.8664	0.585	3.189	0.001	0.719	3.013	
auto_model_X6	1.7902	0.710	2.521	0.012	0.398	3.182	

LOGISTIC REGRESSION MODEL

- Performed Prediction on the.

- Confusion Matrix.

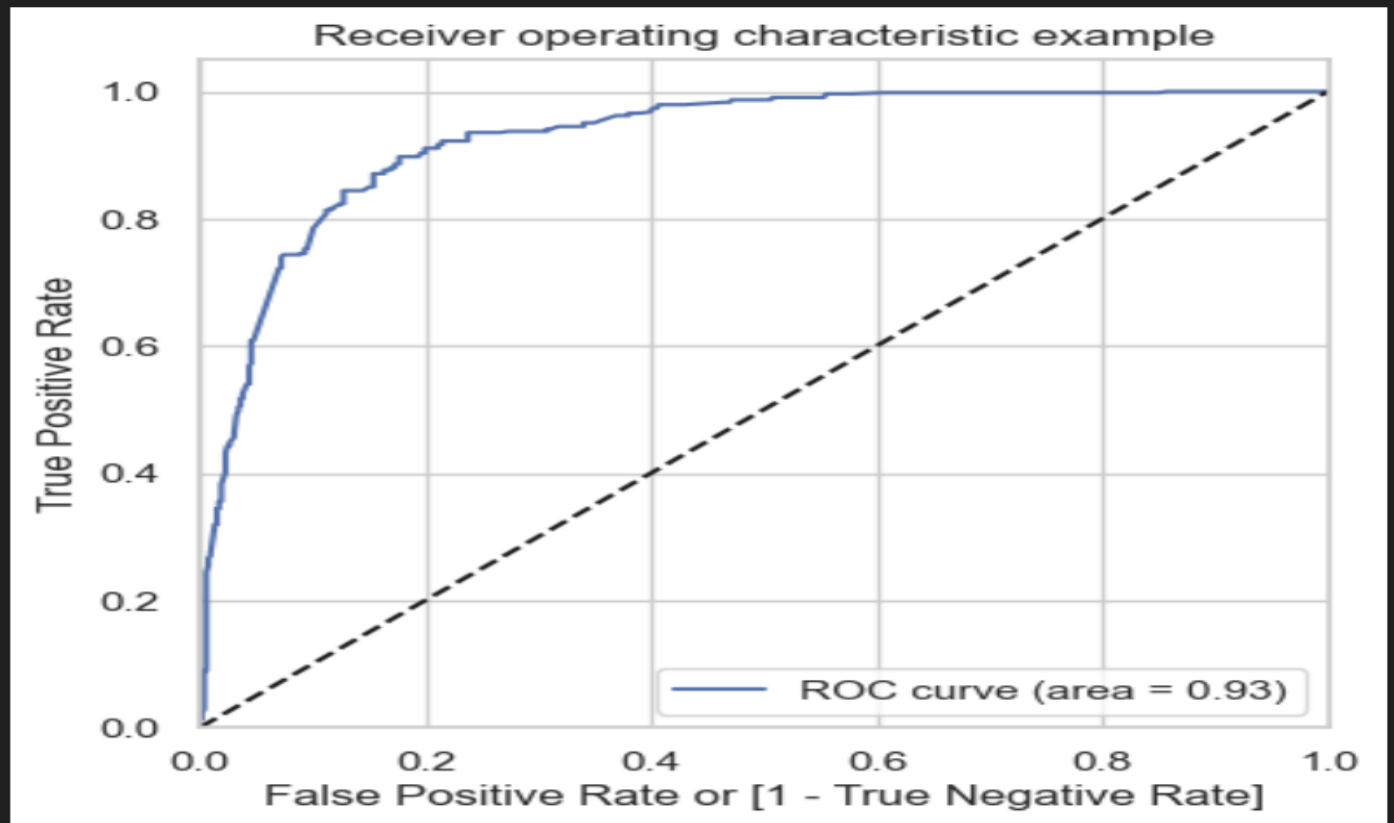
```
array([[445, 81],  
       [ 68, 458]])
```

- Sensitivity, specificity, precision, recall and F1-score.

```
sensitivity: 0.870722433460076  
specificity: 0.8460076045627376  
precision: 0.849721706864564  
recall: 0.870722433460076  
f1 score: 0.8600938967136151
```

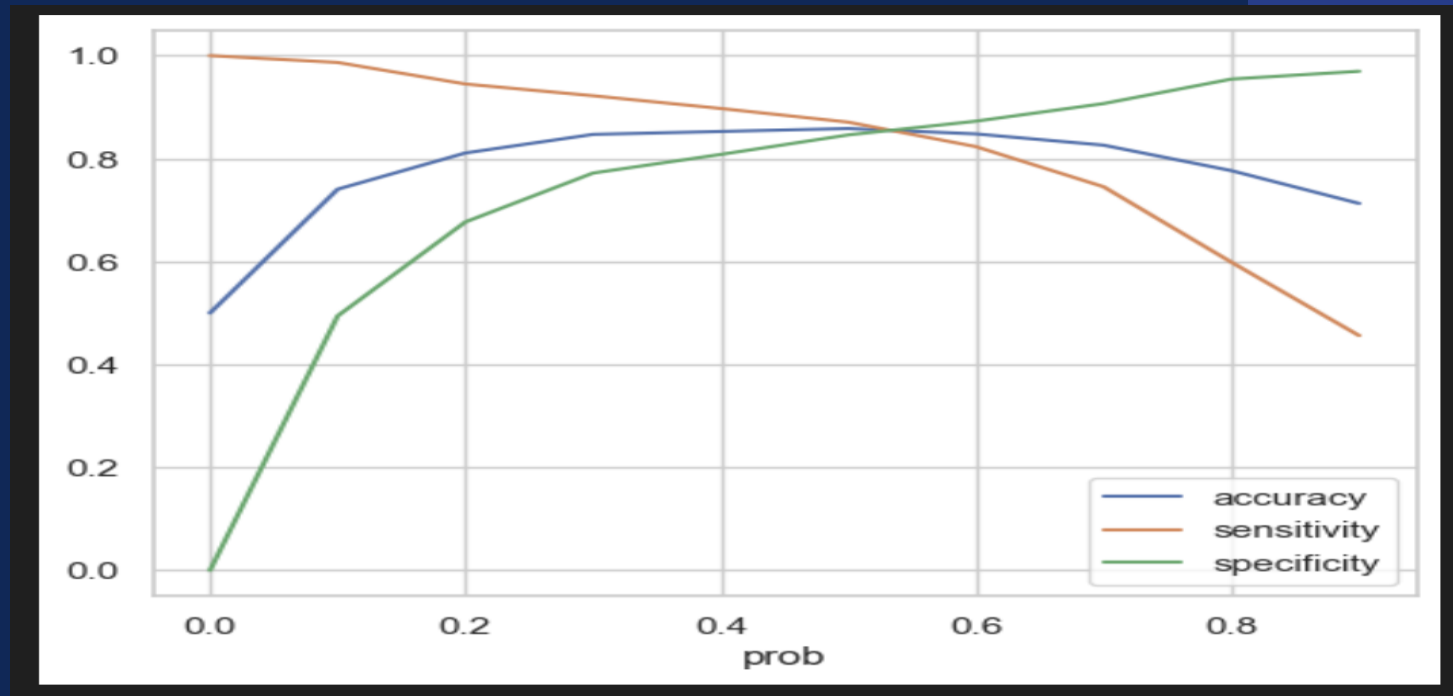
LOGISTIC REGRESSION MODEL

- ROC Curve.



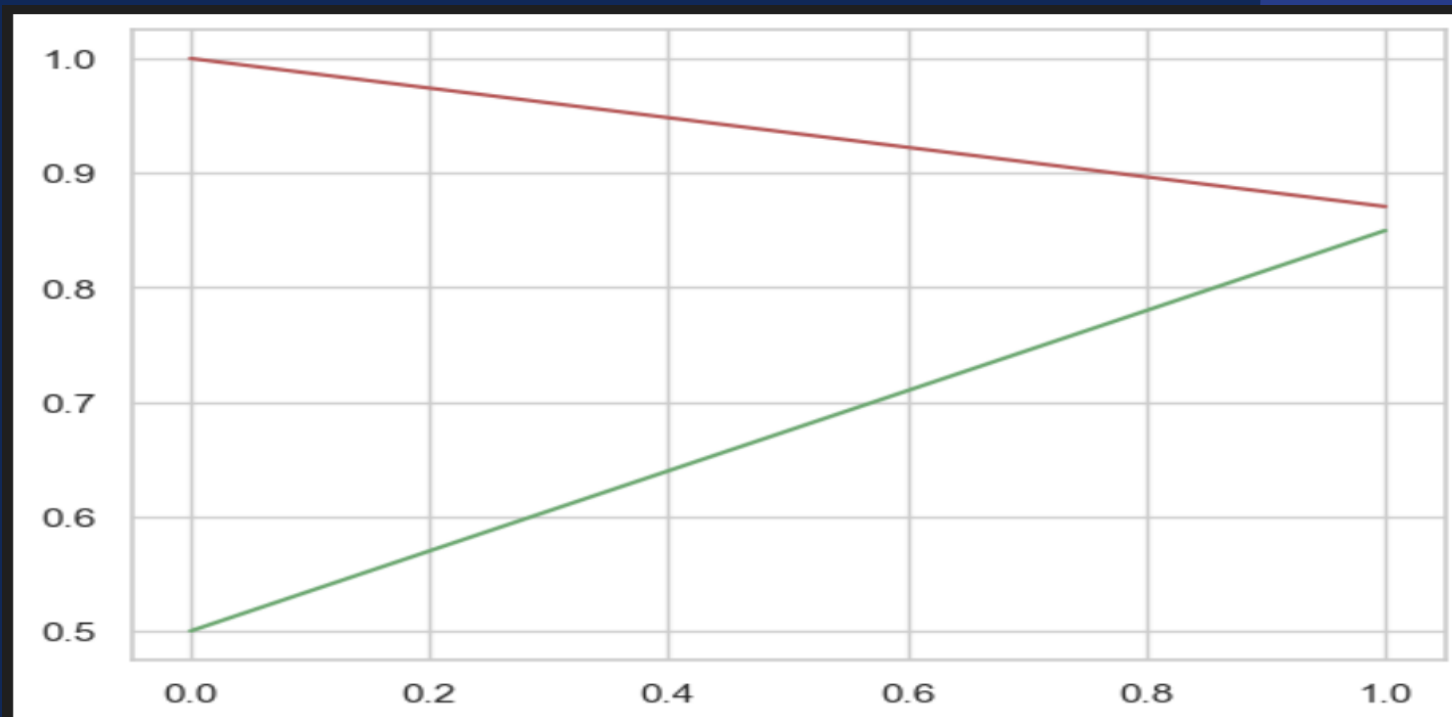
LOGISTIC REGRESSION MODEL

- Optimal cutoff for the model is 0.5.



LOGISTIC REGRESSION MODEL

- Precision Recall Curve.



RANDOM FOREST MODEL

- Used RandomForestClassifier Library from sklearn to build the Model.
- Used the below code.

```
rf = RandomForestClassifier(random_state=42, n_estimators=10, max_depth=3)
rf.fit(X_train_resampled, y_train_resampled)
rf.estimators_[0]
sample_tree = rf.estimators_[4]
```

RANDOM FOREST MODEL

- Selected the important features from the model.
- Used a threshold of 0.01 for shortlisting the features.

```
# Select features with high importance scores
threshold=0.01
selected_rf_features=imp_df[imp_df['Imp'] > threshold]['Varname'].tolist()

selected_rf_features

# Create a new training data with only the selected features
selected_features_df=X_train_resampled[selected_rf_features]
```

RANDOM FOREST MODEL

- Performed the prediction on the training set.
- Accuracy of the model is 85%.
- Confusion Matrix

```
array([[439,  87],  
       [ 71, 455]])
```

RANDOM FOREST MODEL

- Sensitivity is 86%
- Specificity is 83%
- Precision is 84%
- Recall is 86%
- F1 score is 85%
- Cross-validation score

```
Cross-validation scores: [0.7535545  0.86729858 0.82380952 0.84285714 0.87619048]  
Mean CV score: 0.8327420446851728
```

RANDOM FOREST MODEL

- Performed Hyperparameter tuning using GridSearchCV.
- Optimal parameters are:
 - Max_depth: 20
 - Criterion: Entropy
 - Min_samples: 5
 - N_jobs: -1
- Accuracy after building the model using above parameter is 92%.
- Sensitivity: 94%
- Specificity: 89%
- Precision: 90%
- Recall: 94%
- F1 Score: 92%

PREDICTION AND MODEL EVALUATION

- Performed Prediction on Validation Data.
- Logistic Regression Model
 - Took 0.5 as the Optimal cutoff.

Metric	Logistic Regression	Random Forest	Remarks
Sensitivity	71%	74%	Random forest detects slightly more positives
Specificity	82%	84%	Random forest reduces false positives better
Precision	56%	61%	Random forest detects makes more accurate positive predictions
Recall	71%	74%	Random forest stronger at capturing positives
F1 Score	63%	67%	Random forest achieves a better balance between Precision and Recall

SUMMARY

○ Observations from Data Analysis

- Strong associations were found with policyholder hobbies, specific incident types, auto make, and auto model, all of which play a significant role.
- There was an indicator where the months_as_customer value was 0, yet the policy bind date and incident date had a large difference.
- The column _c39 was completely empty.
- High correlation was observed between months_as_customer and age, and also between total_claim_amount and injury_claim.
- The majority of the target class was N compared to Y.

○ Resampling Insights

- Resampling was performed on the training data to address class imbalance in the minority class.
- The RandomOverSampler technique was used to balance the classes.
- After resampling, the dataset contained 526 records for the Y class and 526 records for the N class.

○ Feature Selection

- Used RFECV for selecting the optimal features, resulting in 43 features being retained.
- Built the model using StatsModels, checked the p-values, and eliminated features with p-values greater than 0.05.
- Checked VIF for each feature to verify and address multicollinearity.

SUMMARY

○ Logistic Regression

- Built the model using the optimal features selected in the previous step.
- Achieved an accuracy score of approximately 86% on the training set.
- Achieved an accuracy score of 80% on the testing set.
- Plotted the ROC curve, with the Area Under the Curve (AUC) equal to 93%.
- Plotted the accuracy, sensitivity, and specificity curves to determine the optimal cutoff point.
- The optimal cutoff point was found to be 0.5 in this case.
- The model does not appear to be overfitting, as the accuracy scores for the training and testing sets are close and show no significant disparity.

○ Random Forest

- Built the initial model with all parameters at their default values.
- Selected **22** columns based on a feature importance threshold of **0.01** from the Random Forest Classifier.
- Built the model with the selected columns and achieved an accuracy of **85%** on the training set.
- Performed hyperparameter tuning using **GridSearchCV** to obtain the best estimator.
- The best estimator parameters were: criterion='entropy', max_depth=20, min_samples=5.
- Achieved an accuracy of **92%** on the training set with the best estimator.
- Achieved an accuracy of **82%** on the validation set using the best estimator parameters.
- The significant difference in accuracy between the training and validation sets indicates that the model is overfitting the training data.

CONCLUSION

- Random Forest model(RF) performs slightly better than logistic regression.
- RF Model detects more fraud claims.
- RF reduces false positives better.
- RF achieves a better balance between precision and recall.
- The Random Forest model clearly overfits the training data, as there is a noticeable gap between the training and testing accuracy scores.
- The Logistic Regression model uses 21 features in its final form.
- The Random Forest model uses 22 features in its final form.
- Common important features in both models include: insured_education_level, insured_hobbies, insured_relationship, incident_severity, incident_state, property_damage, and auto_model. These features provide significant insight into the dependent variable.
- Both models can help in identifying fraudulent claims.



THANK YOU

Harshit Kumar Vaish
harshvaish16@gmail.com

Ganesh Babu Kamma
ganeshbabukamma@gmail.com